Mathematics of Multi-Head Attention

This reading will help you understand the mathematics of multi-head attention better. We will walk through some of the computations and give you additional background information.

The Intuition behind Attention

Attention mechanisms allow a model to assign a weight to each position in a sequence. Multi-head attention is based on the same idea, repeated in parallel with separate sets of learnable parameters (*heads*), each focusing on different kinds of relationships.

Attention mechanisms are like a smart note-taking system during a conversation. When you're listening to someone speak, you don't give equal weight to every word – you naturally pay more attention to certain keywords or phrases based on what you're trying to understand. For example, if someone replies "The red car crashed into the blue truck." to your question "What was damaged?", you'll likely focus more on "car" and "truck" than on "red" or "blue."

Multi-head attention takes this idea even further – it's like having multiple listeners who each focus on different aspects of the conversation simultaneously. One head might focus on subject–object relationships (who did what to whom), another might track temporal relationships (what happened when), while a third might focus on descriptive relationships (what properties things have). When all these "listeners" combine their understanding, they create a richer, more nuanced representation of the input. This is why multi-head attention is so powerful – it allows a model to capture different types of relationships in parallel, much like how our own understanding of language comes from simultaneously processing multiple aspects of what we hear or read.

Example Input

For this reading, let's consider a short sentence of 3 tokens: *You found me*

We assume that each token is embedded into a vector in $\mathbb{R}^{d_{\text{model}}}$. In our example, we choose $d_{\text{model}} = 4$. The embeddings can then be represented by the following matrix of shape (3 x 4), with each row representing the embedding of one of the words:

$$X = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{bmatrix}.$$

Computing a Single Attention Head

In multi-head attention, the following computations are done for all heads, but in this example we only compute one head.

1. Linear Projections to Q, K, V

We learn three parameter matrices for this head:

$$W^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

Often, $d_k = d_v = \frac{d_{\text{model}}}{h}$ if there are h heads, but let us pick (for example) $d_k = d_v = 2$ just to illustrate the computation clearly. That means each matrix is (4×2).
We define the following weight matrices:

$$W_1^Q = \begin{bmatrix} 0.1 & 0.2 \\ 0.0 & 0.1 \\ -0.1 & 0.0 \\ 0.2 & 0.1 \end{bmatrix}, \quad W_1^K = \begin{bmatrix} 0.0 & 0.1 \\ 0.1 & 0.0 \\ 0.2 & 0.1 \\ -0.1 & 0.1 \end{bmatrix}, \quad W_1^V = \begin{bmatrix} 0.2 & 0.0 \\ 0.0 & 0.2 \\ -0.1 & 0.1 \\ 0.1 & -0.1 \end{bmatrix}.$$

Then we multiply:

$$\mathbf{Q}_1 = \mathbf{X}\, W_1^Q, \quad \mathbf{K}_1 = \mathbf{X}\, W_1^K, \quad \mathbf{V}_1 = \mathbf{X}\, W_1^V.$$

All resulting matrices are of size (3×2). In our example, we obtain:

$$\mathbf{Q}_1 = \begin{bmatrix} 0.06 & 0.08 \\ 0.14 & 0.24 \\ 0.22 & 0.40 \end{bmatrix}, \quad \mathbf{K}_1 = \begin{bmatrix} 0.04 & 0.08 \\ 0.12 & 0.20 \\ 0.20 & 0.32 \end{bmatrix}, \quad \mathbf{V}_1 = \begin{bmatrix} 0.03 & 0.03 \\ 0.11 & 0.11 \\ 0.19 & 0.19 \end{bmatrix}.$$

2. Scaled Dot-Product Attention

Each head in multi-head attention is computed as:

$$\text{head}_1 = \text{Attention}(\mathbf{Q}_1, \mathbf{K}_1, \mathbf{V}_1) = \text{softmax}\Big(\frac{\mathbf{Q}_1 \mathbf{K}_1^T}{\sqrt{d_k}}\Big)\, \mathbf{V}_1.$$

Here, Q_1 and K_1 are (3×2), so $Q_1\, K_1^T$ is (3×3). We then divide by $\sqrt{2}$ and apply a row-wise softmax. Finally, we multiply by V_1 to get the output shape (3×2).

Dot Products For row i, column j:

$$(Q_1\, K_1^T)_{i,j} = q_i \cdot k_j.$$

We then scale by $\frac{1}{\sqrt{2}}$ and apply:

$$\alpha_{i,j} = \frac{\exp(s_{i,j})}{\sum_{l=1}^{3} \exp(s_{i,l})} \quad \text{where } s_{i,j} = \frac{q_i \cdot k_j}{\sqrt{2}}.$$

Each row's $\alpha_{i,j}$ sums to 1.

Multiply by $\mathbf{V}_1$ Row i of the final attention output is:

$$head_{1_i} = \sum_{j=1}^{3} \alpha_{i,j}\, v_j.$$

Hence, we get a final matrix head_1 of shape (3×2):

$$head_1 = \begin{bmatrix} 0.1105 & 0.1105 \\ 0.1115 & 0.1115 \\ 0.1125 & 0.1125 \end{bmatrix}$$

(If you want to deepen your understanding, try to compute $head_1$ yourself and see if you get the correct result!)

Extending to Multiple Heads

In multi-head attention with $h > 1$ heads:

1. Each head h has its own W_h^Q, W_h^K, W_h^V.
2. Compute the scaled dot-product attention for each head in parallel.
3. Concatenate the resulting outputs along the feature dimension, then apply a final linear projection W^O of size $\big(h \cdot d_v\big) \times d_{\text{model}}$:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}\Big(\text{head}_1, \ldots, \text{head}_h\Big)\, W^O.$$

Each head learns different projections, capturing different relationships in parallel.

The Intuition behind Q, K, and V

Think of using a library's search system. In this analogy:

- The query (Q) is like your search request – what you're looking for; for example, when you type "books about space exploration" into the library catalog.
- The keys (K) are like the tags or index cards in the library catalog that help match your query to relevant materials. Each book has associated keywords and metadata that help the system find matches.
- The values (V) are the actual content or information you want to retrieve – in our analogy, the actual books themselves.

The attention mechanism computes how well each query matches each key (through their dot product), and then uses these match scores to create a weighted combination of the values. To continue our library analogy:

1. You submit your query "space exploration;"
2. The system compares it against all key terms in its catalog;
3. Books that have strongly matching keys (like "NASA," "astronauts," or "rockets") receive higher weights;
4. The system then retrieves and combines information from the matching books, giving more emphasis to the better matches.

In neural networks, this process allows each position in a sequence to "query" all other positions to figure out which parts of the input are most relevant for creating its representation. Multiple heads allow this querying process to happen in parallel, with each head potentially specializing in different types of relationships or patterns in the data.

This is why, in the document example, for the word "You," the model can learn to pay attention to relevant context words like "found" and "me" in different ways through different attention heads, each capturing different aspects of how these words relate to each other.

Final Note: End-to-End Training

In Transformers, multi-head attention is used:

- in the encoder (self-attention),
- in the decoder (self-attention on partially generated output), and
- in encoder–decoder attention (queries from the decoder, keys and values from the encoder).

Typically, we still train with a standard cross-entropy loss. Gradients flow back through the entire multi-head attention machinery, updating W^Q, W^K, W^V, W^O and the token embeddings along with all other model parameters.

Summary

The core algorithm behind multi-head attention in Transformer models is as follows:

1. Embed input tokens into X;
2. Project to Q, K, V;
3. Scaled dot-product attention: $\text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right)\mathbf{V}$;
4. Multiple heads: repeat step 3 for multiple heads; concatenate results;
5. Linear projection back to $\mathbf{X}$'s dimension if necessary;
6. Loss: typically cross-entropy.